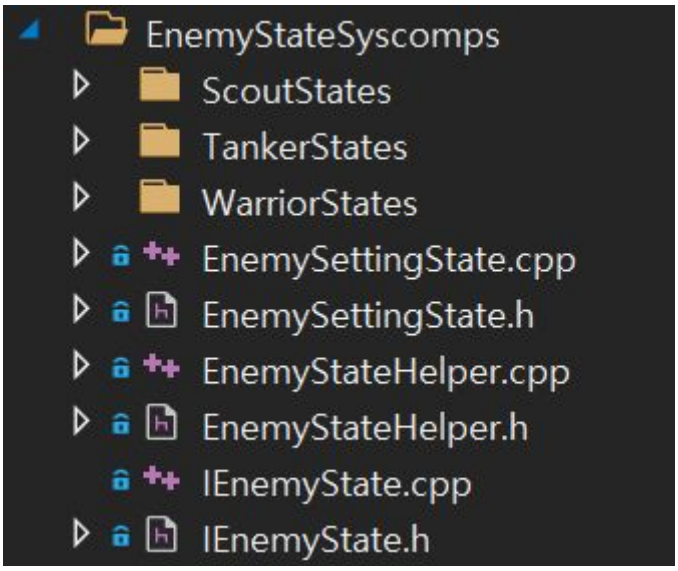
Milestone report

MILESTONE: Pre-Alpha
TEAM NAME: Noflex
GAME NAME: Q
NAME: Minjae Kyung
JOBS: Product Manager
DEGREE PROGRAM: RTIS
GAME DESCRIPTION: Our game is a side view platformer rogue-like game. Player is an eminent scientist who participated in the space project.
LINKS: https://github.com/Rudy-Castan-DigiPen-Teaching/game-project-noflex

Work Done :

- *Restruct Enemy AI*
  - When I was making Enemy AI, my main focus was just to see my algorithm works well.
  - So I did not care about maintainability. so it was in the hell of if-else disease
  - My first goal of this vacation was to fix the structure of the enemy system.
  - To make the code maintainable, I thought that the best solution will state system.
  - It may seem weird to have a state system on components while using an ECS but I can't think of a better way.
  - Thanks to state system, I can add other types of enemies quite easily
  - And I am trying to find another mechanism for pathfinding on platformers.
  - I tried A* path finding. But the main problem was the map is not that difficult and we should give jump information so it does not improve by just having a jump trigger on the platformer. I've also tried ANN training. but I trained quite a lot but haven't made any improvements.- it means it needs a lot of time to train it.




```cpp
template<typename T>  <T> Provide sample template arguments for IntelliSense
class IEnemyState
{
public:
    using IDType = int;

    inline static IDType StateID = q_engine::IDGenerator::GenerateTypeID();
    static std::string StateName; //for Debugging purpose

public:

    static IDType GetStateID() { return StateID; }

    static void Load(q_engine::GameState* , IDType ) {}
    static void Update(q_engine::GameState* ,IDType,  double) {}
    static void Unload(q_engine::GameState* , IDType ) {}

    static void OnTriggerEvent(q_engine::GameState*, IDType, q_engine::TriggerEvent*) {}
    static void OnRayEvent(q_engine::GameState*, IDType, q_engine::RayTriggerEvent*) {}
    static void OnAttackEvent(q_engine::GameState*, IDType) {}
};
```

```cpp
#include "../../../Engine/Component/ObjectStateComponent.h"
#include "../../../Engine/Component/RaycastComponent.h"

#include "../../../Engine/Event/RayTriggerEvent.h"
#include "../../../Engine/Event/TriggerEvent.h"

#include "../../Entity/MapEntity.h"
#include "../../Entity/BombEntity.h"
#include "../../Entity/JumpTriggerEntity.h"
#include "../../Entity/WeaponEntity.h"


bool temp_helper_isInBox(q_engine::StaticRectCollisionComponent rect, glm::vec2 point)
{
    Rect box = rect.GetColliderBox();
    return box.Left() < point.x && point.x < box.Right() && box.Bottom() < point.y&& point.y < box.Top();
}

//temp code
bool isCliffWall(q_engine::GameState* controlledState, IDType enemyID)
{
    Rect enemyBox = controlledState->GetComponents<q_engine::RectCollisionComponent>()[enemyID].GetColliderBox();
    EnemyStateComponent& enemyState = controlledState->GetComponents<EnemyStateComponent>()[enemyID];

    glm::vec2 enemyCliffCheckPoint = enemyState.isMovingRight() ? glm::vec2{ enemyBox.Right() + 2.f, enemyBox.Bottom() - 1.f } : glm::vec2{ enemyBox.Left() - 2.f, enemyBox.Bottom(
    glm::vec2 enemyWallCheckPoint1 = enemyState.isMovingRight() ? glm::vec2{ enemyBox.Right() + 2.f, enemyBox.Bottom() + 1.f } : glm::vec2{ enemyBox.Left() - 2.f, enemyBox.Bottom()
    glm::vec2 enemyWallCheckPoint2 = enemyState.isMovingRight() ? glm::vec2{ enemyBox.Right() + 2.f, enemyBox.Top() - 1.f } : glm::vec2{ enemyBox.Left() - 2.f, enemyBox.Top() - 1.f

    bool isCliff = true, isWall = false;

    for (auto& collisionBox : controlledState->GetComponents<q_engine::StaticRectCollisionComponent>())
    {
        /*
        if (collisionBox.GetEntityTypeID() != q_engine::GetTypeID<q_engine::MapEntity>())
            continue;
        */

        if (temp_helper_isInBox(collisionBox, enemyWallCheckPoint1))
            isWall = true;
        if (temp_helper_isInBox(collisionBox, enemyWallCheckPoint2))
            isWall = true;

        if (temp_helper_isInBox(collisionBox, enemyCliffCheckPoint))
            isCliff = false;
    }


    return isCliff || isWall;
}

glm::vec2 temp_NuckBack{ 200, 600};

void BasicEnemyTriggerEvent(q_engine::GameState* controlledState, IDType enemyID, q_engine::TriggerEvent* triggerEvent)
{
    IDType CollidedID = triggerEvent->mTriggerEntityID;

    auto& enemyState = controlledState->GetComponents<EnemyStateComponent>()[enemyID];
    auto& enemyTransform = controlledState->GetComponents<q_engine::TransformComponent>()[enemyID];

    if (triggerEvent->mTriggerEntityTypeID == q_engine::GetTypeID<WeaponEntity>())
    {
        auto& weaponTransform = controlledState->GetComponents<q_engine::TransformComponent>()[CollidedID];
        auto& weaponComponent = controlledState->GetComponents<WeaponComponent>()[CollidedID];

        if (!weaponComponent.IsAttacking())
            return;


        glm::vec2 relativeTransform = weaponTransform.GetPosition() - enemyTransform.GetPosition();
        if (relativeTransform.x > 0)
            enemyState.Jump({ -temp_NuckBack.x,temp_NuckBack.y });
        else
            enemyState.Jump(temp_NuckBack);

        auto& enemyHP = controlledState->GetComponents<HealthComponent>()[enemyID];
        enemyHP.ChangeCurHealth(-BombEntity::DAMAGE);
    }
    else if (triggerEvent->mTriggerEntityTypeID == q_engine::GetTypeID<BombEntity>())
    {
        auto& BombTransform = controlledState->GetComponents<q_engine::TransformComponent>()[CollidedID];

        glm::vec2 relativeTransform = BombTransform.GetPosition() - enemyTransform.GetPosition();
        if (relativeTransform.x > 0)
            enemyState.Jump(2.f * glm::vec2{ -temp_NuckBack.x,temp_NuckBack.y });
        else
            enemyState.Jump(2.f * temp_NuckBack);

        auto& enemyHP = controlledState->GetComponents<HealthComponent>()[enemyID];
        enemyHP.ChangeCurHealth(-BombEntity::DAMAGE);
    }
}

void EnemyJumpTriggerEvent(q_engine::GameState* controlledState, IDType enemyID, q_engine::TriggerEvent* triggerEvent)
{
    if (triggerEvent->mTriggerEntityTypeID != q_engine::GetTypeID<JumpTriggerEntity>())
        return;

    auto& enemyState = controlledState->GetComponents<EnemyStateComponent>()[enemyID];
    auto& enemyTransform = controlledState->GetComponents<q_engine::TransformComponent>()[enemyID];

    auto& JumpTrigger = controlledState->GetComponents<JumpTriggerComponent>()[triggerEvent->mCollideWithEntityID];


    glm::vec2 relativePosition = enemyState.GetLastSeenPlayerPosition() - enemyTransform.GetPosition();


    if (isBigger(enemyState.GetMaxJumpVelocity(), JumpTrigger.GetRequireVelocity()))
    {
        if (JumpTrigger.isToUpper() && relativePosition.y > 0)
            enemyState.Jump(JumpTrigger.GetRequireVelocity());
        else if (!JumpTrigger.isToUpper() && relativePosition.y <= 0)
            enemyState.Jump(JumpTrigger.GetRequireVelocity());
    }
}

double cross(const glm::vec2& thisPtr, const glm::vec2& other)
{
    return thisPtr.x * other.y - thisPtr.y * other.x;
}

bool isPlayerOnSight(q_engine::GameState* controlledState, IDType enemyID, q_engine::RayTriggerEvent* triggerEvent)
{
    auto& RayComponent = controlledState->GetComponents<q_engine::RaycastComponent>()[enemyID];
    auto& enemyState = controlledState->GetComponents<EnemyStateComponent>()[enemyID];

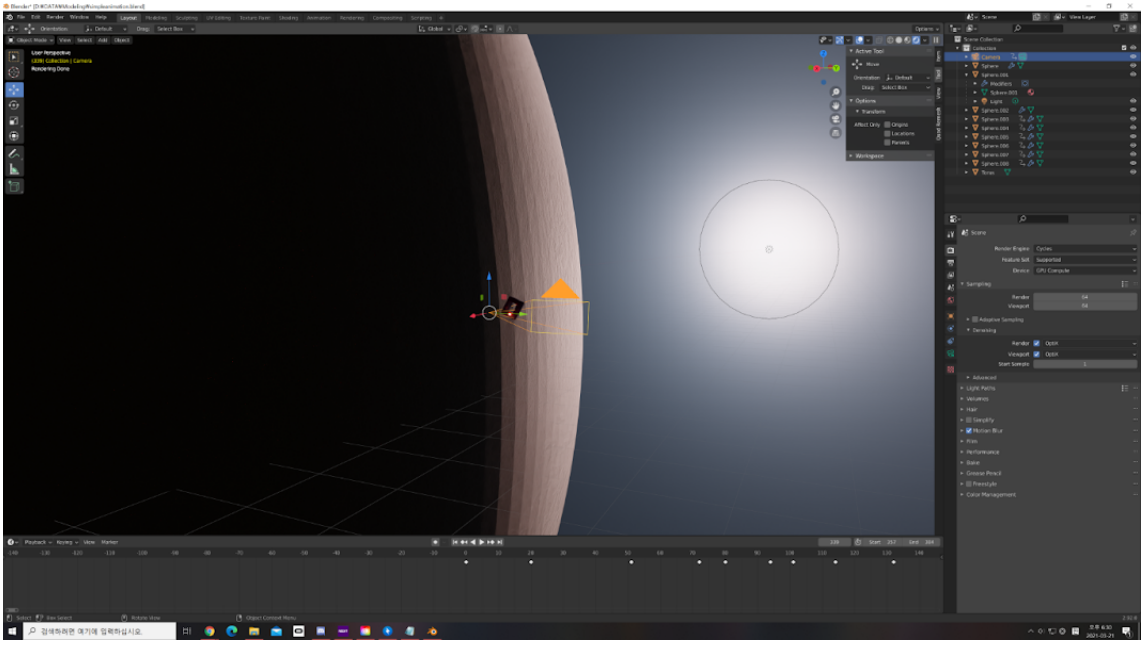
    if (triggerEvent->rayColided.typeID == q_engine::GetTypeID<PlayerEntity>()
        && RayComponent.GetDistanceData().at(triggerEvent->rayIndex) < enemyState.GetSightDistance())
    {

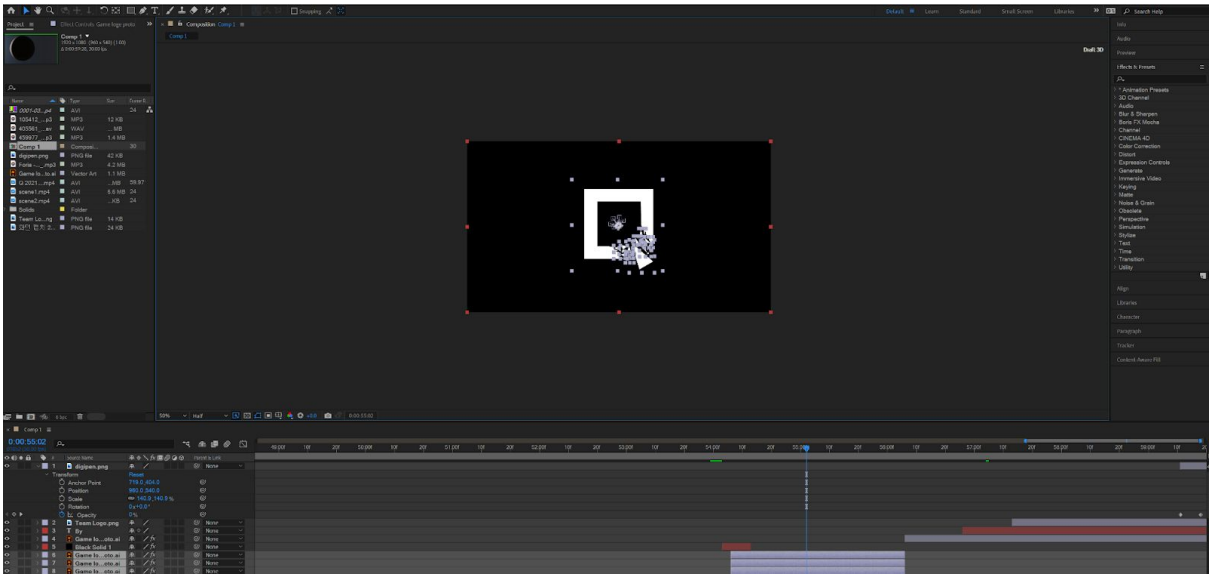
        auto& playerTransform = controlledState->GetComponents<q_engine::TransformComponent>()[triggerEvent->rayColided.entityID];
        auto& enemyTransform = controlledState->GetComponents<q_engine::TransformComponent>()[enemyID];

        glm::vec2 relativePosition = playerTransform.GetPosition() - enemyTransform.GetPosition();

        glm::vec2 upFov = glm::vec2{ cos( enemyState.GetLookingDirection() + enemyState.GetSightFOV()), sin( enemyState.GetLookingDirection() + enemyState.GetSightFOV())};
        glm::vec2 downFov = glm::vec2{ cos( enemyState.GetLookingDirection() - enemyState.GetSightFOV()), sin( enemyState.GetLookingDirection() - enemyState.GetSightFOV())};
```

- *Pre-alpha Trailer*
    - I make a trailer for pre-alpha
    - https://drive.google.com/file/d/1W11rC0E7dvba9mbRLOaTcZMnT0sKbMLy/view?usp=sharing
    - I was trying to add some 3D motion graphics. It was quite unsatisfactory.
    - I want to add more cool transitions to the trailer but due to bugs on After Effect, I can't.





Notes :